Julius-Maximilians-
UNIVERSITÄT
WÜRZBURG

Chair of Computer Science VIII
Aerospace Information Technology
Prof.Dr.Sergio Montenegro.

Exercise:

(1) RODOS HAL interface for GPIO
(2) Timer/Counter in Stm32f407

Contents

1 RODOS:PRINTF **2**

2 RODOS Hardware Abstraction Layer (HAL) for GPIO **6**

3 Background of Timer/Counter **7**

4 Timer Counter in Stm32f407 Discovery **7**

4.1 **Task3**: Blinking LED using Timer. 8

4.2 **Task4**: Timer Interrupt 8

1 RODOS:PRINTF

- You can print out data with the help of PRINTF function that writes data to serial port.
  - This PRINTF works exactly the same way as the printf in C. e.g
  - PRINTF("\n Hello World from Student no %d \r",variable);
    - \n is used to print the statement from new line.
    - \r is used to bring the cursor on the start of the line.
- To use this PRINTF first connect the USB–To–UART adapter to your board as shown below.

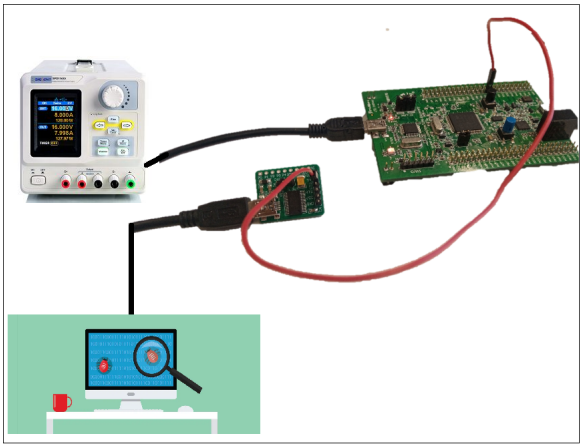


Figure 1: Debugging Using Serial Port by "PRINTF" function

- As soon as you connect the second USB between USB–To–UART adapter and your PC, you should observe following in your Virtual Box Manager:

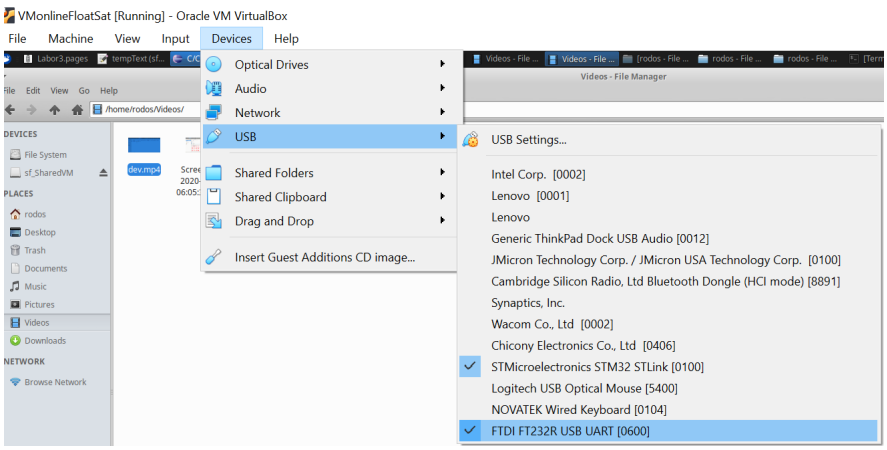


Figure 2: ftdi connection

- The Tx of USB–To–UART adapter pin is connected to PD5 pin of the discovery board.
  - It is done because in the RODOS project if you go to the file:
  - /eclipse_ws2020/rodos/src/bare–metal/stm32f4/platform–parameter/discovery/platform–parameter.h OR in Windows
  - \Eclipse2019\Workspace\rodos\src\baremetal\stm32f4\params.h

```
#ifndef UART_DEBUG
//#define UART_DEBUG                    UART_IDX3, GPIO_056, GPIO_057 // PD8 and PD9
#define UART_DEBUG                    UART_IDX2, GPIO_053, GPIO_054 // PD5 and PD6
//#define UART_DEBUG                    UART_IDX4, GPIO_000, GPIO_001 // PA0 and PA1
//#define UART_DEBUG                    UART_IDX5
#endif
```

Figure 3: platform–parameter.h or params.h file

- You have been provided a serial monitor Hterm
- If you go to run configuration in eclipse there you can start the Hterm utility as shown below:

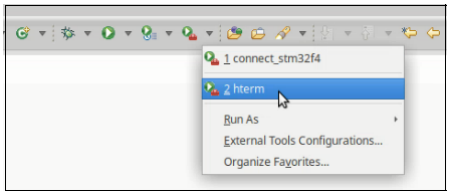


Figure 4: Hterm Serial Monitor

- This utility has a connection tab where you can set the COM port and Baud rate for serial communication.

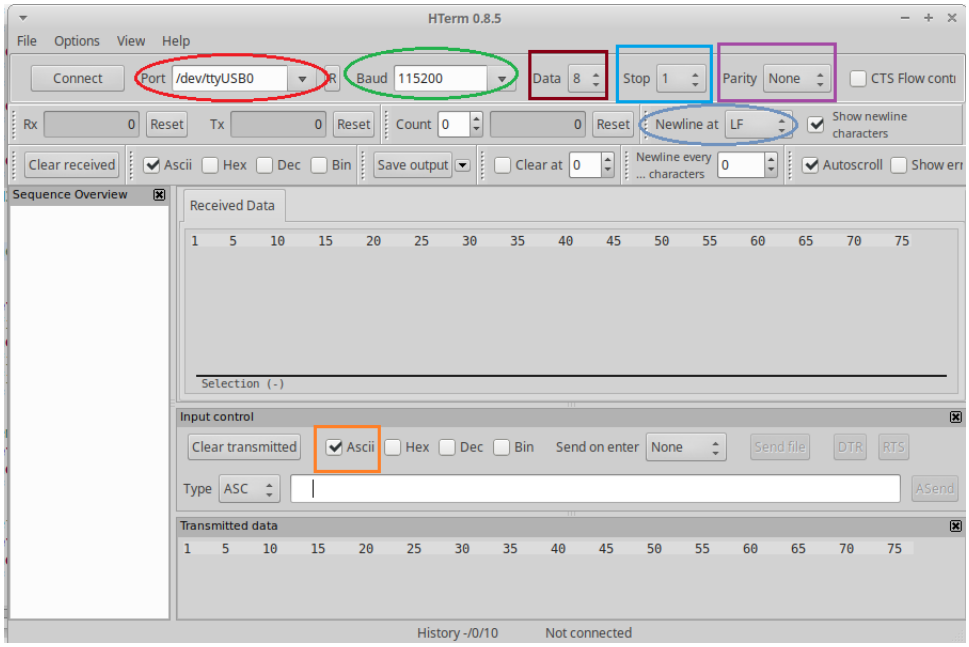


Figure 5: Hterm Serial Monitor

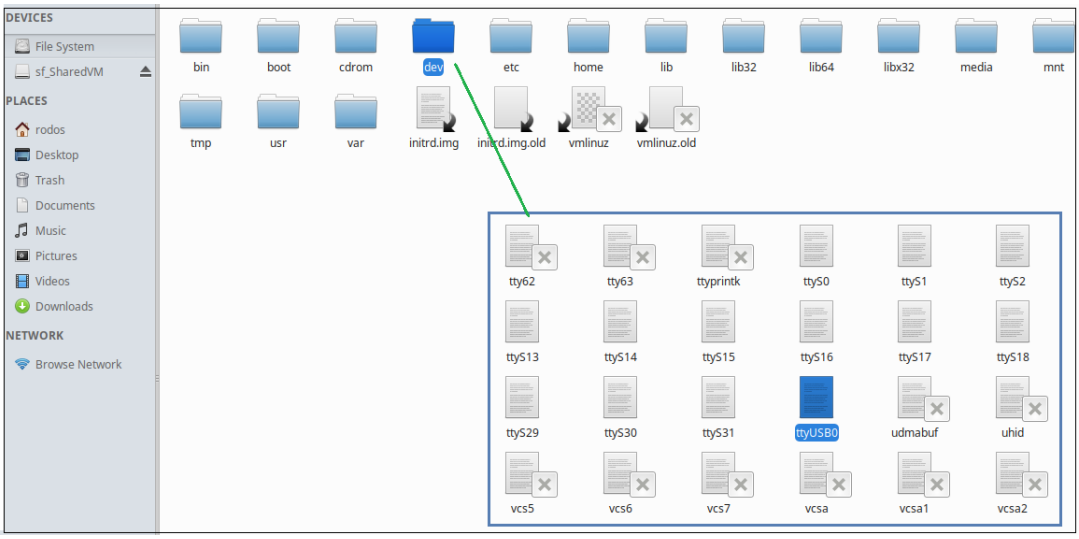


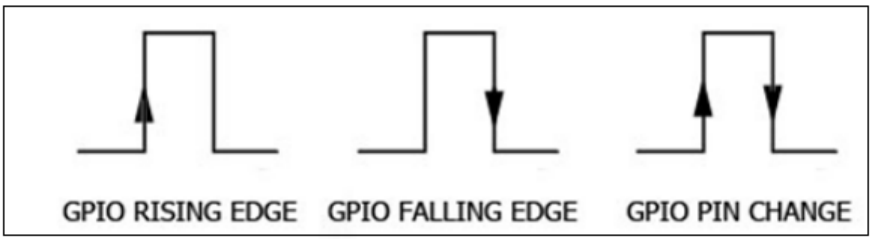
Figure 6: Available Serial Ports in Linux

- **Task1**:Use the RODOS's PRINTF function to print the following:
  - char a[20]= "Hello World";
  - float pi=3.41222;
  - long nano_second=640040;

- Observe the output in Hterm utility.

2 RODOS Hardware Abstraction Layer (HAL) for GPIO

- **Task2**:Implement a project in which you can measure the width of signal pulses with following requirements:
  1. The available User Button on the development board should be use to generate signal pulses on PA0.
  2. The thread should be suspended until a rising or falling edge event is detected using suspendUntilDataReady() function.
  3. The width of the measured signal in milliseconds should be printed out every time a signal pulse is generated on PA0.



Note:

- Use the config function of HAL_GPIO class for the sensitivity configuration (IRQ sensitivity):*Have a look on the HAL_GPIO class or the second last slide of exercise 1*.
- You should use the interruptEnable(int) function of HAL_GPIO class
- Use resetInterruptEventStatus() function to reset the interrupt event status every time an event has occurred
- Use NOW() function which returns the time now in the internal time resolution (Nanoseconds).

3 Background of Timer/Counter

Counter/timer hardware is a crucial component of most embedded systems. In some cases a timer is needed to measure elapsed time, In others we want to count or time some external events. Basically timer is a clock that controls an event sequence at fixed amount of time. Timers are used for the precise delay generation and also used to trigger an activity before and after a predetermined time and to measure the time elapsed between two successive events

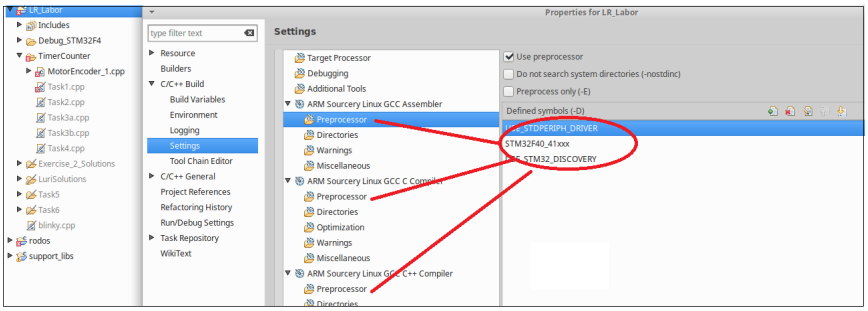
4 Timer Counter in Stm32f407 Discovery

The STM32F4 device includes two advanced-control timers, ten general-purpose timers, two basic timers, plus two watchdog timers and one SysTick timer:

Timer type	Timer	Counter Resolution	Counter type	Prescaler Factor	DMA Request Generation	Capture/ Compare Channels	Complementary Output	Interface Clock (TIMx_CK_INT)	Max Timer Clock (MHz)
Advanced Control	TIM1, TIM8	16-bit	Up, Down, Up/down	between 1 and 65536	Yes	4	Yes	APB2 Timer Clock	168
General Purpose	TIM2, TIM5	32-bit	Up, Down, Up/down	between 1 and 65536	Yes	4	No	APB1 Timer Clock	84
	TIM3, TIM4	16-bit	Up, Down, Up/down	between 1 and 65536	Yes	4	No	APB1 Timer Clock	84
	TIM9	16-bit	Up	between 1 and 65536	No	2	No	APB2 Timer Clock	168
	TIM10, TIM11	16-bit	Up	between 1 and 65536	No	1	No	APB2 Timer Clock	168
	TIM12	16-bit	Up	between 1 and 65536	No	2	No	APB1 Timer Clock	84
	TIM13, TIM14	16-bit	Up	between 1 and 65536	No	1	No	APB1 Timer Clock	84
Basic	TIM6, TIM7	16-bit	Up	between 1 and 65536	Yes	0	No	APB1 Timer Clock	84

Advanced-control timers (TIM1, TIM8):
The advanced-control timers TIM1, and TIM8 can be seen as three-phase PWM generators multiplexed on 6 channels.
General-purpose timers (TIMx):
TIM2, TIM3, TIM4, and TIM5 are full-featured general-purpose timers. They all feature 4 independent channels for Input Capture, Output Compare, PWM or one-pulse mode output.
Independent Watchdog:
The independent watchdog is based on a 12-bit downcounter and 8-bit prescaler. It is clocked from an independent 32 kHz internal RC and as it operates independently from the main clock, it can operate in Stop and Standby modes.
Window Watchdog:
The window watchdog is based on a 7-bit downcounter that can be set as free-running. It can be used as a watchdog to reset the device when a problem occurs. It is clocked from the main clock.
SysTick Timer:
This timer is dedicated to real-time operating systems, but could also be used as a standard downcounter.

- Before Doing any exercise regarding the timer please make sure that you have following lines in the compile configurations of both **rodos** and **LR_Labor** project.
  - USE_STDPERIPH_DRIVER
  - STM32F40_41xxx
  - USE_STM32_DISCOVERY



4.1 Task3: Blinking LED using Timer.

- Modify the blinking frequency in Timer_Template.cpp file to 0.5Hz, 2Hz, 4Hz

Timer's Key Parameters There are two most important parameters to configure the onboard timers in a microcontrollers.

1. Prescaler: Specify how fast the timer will count.

   - Prescaler is a kind of slow down for Timer incrementing, instead of incrementing each clock cycle it can be adjusted to every 2nd, 4th.,8th. etc.
   - You can set the Frequency_After_Prescaler depending on the acceptable prescalar value according to the table given in the exercise.

$$\boxed{Prescaler = \frac{Timer_Max_Frequency}{Frequency_After_Prescaler} - 1}$$

2. Period: Determines the frequency of interrupt depending on the desired frequency.

$$\boxed{Period = \frac{Frequency_After_Prescaler}{Timer_Desired_Frequency} - 1}$$

4.2 Task4: Timer Interrupt

- Generate interrupt every given interval using timer on STM32F4 Discovery board.
- Make green LED toggle every 500ms interval using TIM2.
- In the main program toggle red LED every 3000ms (red LED toggling will not happen using timer interrupt, but just by simple delay)

To perform this task during the Timer Initialization you also need to configure the Nested Vector Interrupt controller (NVIC) which will handle the Timer interrupt, The NVIC settings are performed as follows:

```
NVIC_InitTypeDef NVIC_InitStruct;
NVIC_InitStruct.NVIC_IRQChannel = TIMx_IROn;
NVIC_InitStruct.NVIC_IRQChannelPreemptionPriority =  ;
NVIC_InitStruct.NVIC_IRQChannelSubPriority =  ;
NVIC_InitStruct.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init(&NVIC_InitStruct);
```

- TIM2 interrupt is most important that's why PreemptionPriority and SubPriority must be zero.

As this task is related to the Timer interrupt you will have to write the code for the interrupt handler that will be executed each time when the timer interrupt triggers. This handler is also called interrupt service routine (ISR) for timer. You have to toggle the green LED right there in this routine as shown below:

```
void TIMx_IRQHandler()
{
    // Checks whether the Timer x interrupt has occurred or not
    if (TIM_GetITStatus(TIMx, TIM_IT_Update))
    {
        // Perform the necessary action.

        // Clears the TIMx interrupt pending bit so that it is available in the next interrupt
        TIM_ClearITPendingBit(TIMx, TIM IT_Update);
    }
}
```